

SiteNetwork — The Whole Thing, Explained Simply

*A plain-English guide to what you built, why every piece exists, and everything we set up together.
Written for someone with basic IT/SEO knowledge — no computer-science degree required.*

Part 1 — What is this, really?

Imagine a **factory that builds websites**. You press a button, and out comes a finished, fast, SEO-optimized website — with its own database, its own admin panel, its own domain, hosted on the internet. Then you can press another button to build a *second* website, a *third*, a hundredth... all from the same factory.

That factory is **SiteNetwork** (the code project is called `pinterest-cms`).

Two ways to think about it:

- **Like Shopify, but for content/SEO sites.** Shopify lets anyone spin up an online store without coding. SiteNetwork lets anyone spin up a content website (blog, local business, news, shop) without coding.
- **Like WordPress multisite on steroids.** One system runs *many* separate websites, but unlike WordPress each site is genuinely isolated (its own database) and is static-fast (no slow PHP on every visit).

The key trick that makes it special: the sites don't live on *your* servers forever. When someone signs up, the system creates the website ****inside that customer's own GitHub and Cloudflare accounts****. They own every file and every bit of hosting. You (the platform) just orchestrate it. That's the "BYO" (Bring Your Own infrastructure) model — it keeps your costs near zero and gives customers real ownership.

Part 2 — The two different "websites" (don't mix these up)

This is the #1 thing that causes confusion, so let's nail it first.

	The Control Panel (the "brain")	The Customer Sites (the "products")
What it is	The dashboard where people sign up and manage things	The actual finished websites visitors see
Where it runs	ONE Cloudflare Worker (your <code>pinterest-cms</code>)	Each in the <i>customer's own</i> Cloudflare account

	The Control Panel (the "brain")	The Customer Sites (the "products")
The code	This whole repo (pinterest-cms)	Built from the site-template repo
Lives at	app.freecoin.slink.de	e.g. f30imask.com , apmc.com.pk , etc.
Tech	Cloudflare Workers + Hono + Turso databases	Astro (a static-site builder)

So the repo you've been deploying ([pinterest-cms](#)) is the **brain**. The [site-template](#) repo is the **cookie-cutter** the brain uses to stamp out each customer site. That's why we spent so long getting [site-template](#) created — without the cookie-cutter, the factory can't stamp out cookies.

Part 3 — The services (accounts) and why each one exists

Your platform is glued together from a handful of online services. Here's each one, in plain terms:

Service	What it does for you	Why this one
Cloudflare Workers	Runs the brain's code, worldwide, instantly	Free tier is generous; code runs "at the edge" (close to users) so it's fast
Turso (libSQL/SQLite)	The databases — one "master" list of sites + one private database per site	Cheap, fast, and lets each site have its <i>own</i> isolated database
Cloudflare R2	Stores uploaded images/files	Like Amazon S3 but with no bandwidth fees
GitHub	Stores the code for every customer site + runs their builds	Every customer site is a GitHub repo they own; GitHub Actions builds it
Resend	Sends emails (verify account, password reset, form alerts)	Simple email API; sends from your domain
Your domain freecoinslink.de	The address of the control panel	You own it; the dashboard lives on app. and customer CMSes on cms.
Cloudflare Analytics Engine	Stores privacy-friendly visitor stats	Built into Workers, super cheap, no cookies needed

Part 4 — Every secret & token we created (and WHY)

You asked specifically about the tokens and their permissions. A **token** (or **secret**) is basically a password that lets one computer act on your behalf on another service. We store these as **Cloudflare Worker secrets** (encrypted, never visible in the code). Here's the complete list and the reason for each.

Golden rule you learned the hard way: never paste a real token into a chat, email, or screenshot. If one leaks, revoke it and make a new one. We had to rotate your Turso token and a GitHub token for exactly this reason.

4a. The brain's own secrets (set with `wrangler secret put`)

Secret name	What it unlocks	Why it needs it
<code>TURSO_MASTER_URL</code> / <code>TURSO_MASTER_TOKEN</code>	The master database	To look up which site a visitor's hostname belongs to
<code>TURSO_API_TOKEN</code>	Turso's management API	To create a brand-new database every time a site is provisioned
<code>JWT_SECRET</code>	Signs admin login cookies	So logged-in CMS sessions can't be forged
<code>SAAS_JWT_SECRET</code>	Signs <i>customer</i> (dashboard) login cookies	Separate from the CMS one, so the two auth systems stay independent
<code>NETWORK_ADMIN_KEY</code>	The "super admin" master key	Lets <i>you</i> provision/manage the whole network
<code>VAULT_MASTER_KEY</code>	The encryption root for the credential vault	So customers' GitHub/Cloudflare tokens are stored encrypted , not in plain text
<code>GITHUB_APP_ID</code> + <code>GITHUB_APP_PRIVATE_KEY</code>	Your GitHub "App" identity	Lets the platform create repos and push code into customers' GitHub accounts
<code>CF_API_TOKEN</code> + <code>CF_ACCOUNT_ID</code> + <code>CF_ZONE_ID</code>	Your Cloudflare account	For DNS + cache-purge automation on your own domain
<code>RESEND_API_KEY</code>	Sending email	Verification + reset + form emails (optional — without it, emails just log)

4b. The GitHub Personal Access Token (the one that kept failing)

This is the token you make at github.com/settings/tokens. We use it *once*, locally on your Mac, to push the `site-template` files up to GitHub. It needs **two** permission boxes ticked:

Scope (checkbox)	Why it's required
<code>repo</code>	To create the repository and write normal files into it
<code>workflow</code>	The template contains <code>.github/workflows/*.yaml</code> files (the build scripts). GitHub <i>refuses</i> to let a token touch workflow files unless this box is ticked. This is the exact thing that blocked us repeatedly — the token had <code>repo</code> but not <code>workflow</code> .

The fix: use the pre-filled link

`github.com/settings/tokens/new?scopes=repo,workflow` so **both** boxes are already ticked** and you can't forget one.

4c. The customer's Cloudflare token (needed later, for deploying their sites)

When a customer connects *their* Cloudflare account, the platform asks them to create a token with a **specific, minimal** set of permissions — only what's needed, nothing more. This is the exact list the code requires (from `src/modules/connections/cloudflare.ts`):

Permission	Access	Why
Account · Workers Scripts	Edit	Deploy the customer's site to <i>their</i> Workers
Account · Account Settings	Read	Read the account id
Zone · Zone	Read	See their domains
Zone · Zone Settings	Edit	Turn on the right settings (SSL, etc.)
Zone · DNS	Edit	Point the domain at the site
Zone · Cache Purge	Purge	Clear the cache when content changes
Zone · Analytics	Read	Show traffic stats
Zone · Firewall Services	Edit	Manage the AI-crawler / bot-protection rules
Zone · Email Routing Rules	Edit	Enable the Site Mailbox feature

That's a "least-privilege" design: the token can do its job but can't, say, delete the customer's other websites.

Part 5 — The configuration variables (the non-secret settings)

These live in `wrangler.toml` in plain text (they're not secret). They're the knobs that tell the brain how to behave. The important ones for your setup:

Variable	Your value	Meaning
<code>SAAS_MODE</code>	<code>"1"</code>	Turns the whole SaaS dashboard ON (without this, signup doesn't work)
<code>SAAS_APP_HOSTNAME</code>	<code>app.freecoin slink.de</code>	Where the dashboard lives
<code>NETWORK_ADMIN_HOSTNAME</code>	<code>pinterest-cm s...workers.de v</code>	The super-admin-only address
<code>SAAS_CMS_HOST_SUFFIX</code>	<code>cms.freecoin slink.de</code>	Each site's CMS gets a <code>something.cms.freecoinslink.de</code> address
<code>SAAS_TEMPLATE_REPO</code>	<code>ArsalR/site- template</code>	The cookie-cutter repo every site is cloned from

Variable	Your value	Meaning
GITHUB_APP_SLUG	sitenetwork-os	The public name of your GitHub App (used in the install link)
SAAS_PBKDF2_ITERATIONS	(empty → 100000)	Password-hashing strength
SAAS_EMAIL_FROM	(empty → default)	The "From" address on emails

Important lesson we learned:** because these are in `wrangler.toml`, every time you run `git pull` and re-deploy, the file's values **overwrite** whatever you typed by hand. That's why your `GITHUB_APP_SLUG` kept resetting to empty and GitHub kept showing "unavailable." The permanent fix was to *write the correct value into the file itself**** (done in PR #103), so pulls and deploys stop wiping it.*

Part 6 — What lives in the code (module by module, plain English)

The brain's code is organized into **modules** — each is a self-contained feature folder under `src/modules/`. Think of them like apps within the app.

Here's what each one does.

The core plumbing — `src/lib/` (shared building blocks)

File	What it does
<code>types.ts</code>	The master list of every setting/secret and shared data shapes
<code>auth.ts</code>	Turns a password into a secure hash (PBKDF2) and back-checks it
<code>apiAuth.ts</code>	Checks API keys on the public REST API
<code>turso.ts</code>	Connects to the right database for each request
<code>provision.ts</code>	The recipe for creating a brand-new site database with all its tables
<code>migrate.ts</code>	Upgrades existing site databases when you add new features (v1–v21)
<code>errors.ts</code>	The fixed list of error codes the API can return (never change these)
<code>idempotency.ts</code>	Stops a repeated request from doing the same thing twice
<code>rateLimit.ts</code>	Stops anyone from hammering the API too fast
<code>webhooks.ts</code>	Sends signed "something happened" notifications to other systems
<code>seo.ts, slugs.ts, redirects.ts, revalidate.ts</code>	SEO helpers, URL slugs, redirects, cache refresh
<code>r2.ts, imageMeta.ts</code>	File storage + reading image dimensions
<code>utils.ts</code>	Odds and ends (ID generation, HTML escaping for safety)

The SaaS layer — `src/modules/` (the platform features)

Module	In one sentence
customers	Sign-up, login, email verification, the dashboard shell
auth	The gate that checks a customer is logged in
app	The dashboard's web pages and the <code>/api/saas</code> endpoints
connections	The wizard where customers connect their GitHub + Cloudflare (with the exact token recipes)
vault	Encrypts and stores those connected credentials safely
provisioning	The step-by-step pipeline that actually builds a new site (the part that was failing at "Create the repository")
sites	"Describe your site and we'll build it" (AI genesis), rollback, site management
design	The visual presets (colors, fonts, layouts) customers pick from
cloning	One-click "make another site like this one"
seo	The big one (28 files): per-post SEO cockpit, analysis, imports from Yoast/RankMath
publishing	The gate that checks quality <i>before</i> a post goes live, then triggers a rebuild
quality-gate	The actual quality-scoring engine (blocks thin/spammy pages)
pseo	Programmatic SEO: paste a spreadsheet + a template → generate many pages safely
linking	Suggests internal links between related posts
importer	Import an existing WordPress site (reads its export file)
forms	Contact/lead forms + automation when someone submits
mail	"Site Mailbox" — receive email at the site's domain
integrations	Scoped API keys + webhooks so tools like n8n/Zapier can plug in
analytics	Privacy-friendly visitor stats + Core Web Vitals
marketing	Ad/marketing pixel management (with consent)
network	The "network brain": Google Search Console, content-decay detection
pinterest	Pinterest traffic engine (OAuth + posting pins)
ecommerce	Optional selling: Stripe checkout + orders
affiliate	Affiliate-link compliance (disclosures, <code>rel</code> tags)
agency	White-label / client seats / monthly reports for agencies
billing	Your own subscription plans (Starter \$29 / Agency \$79)
webhooks	The bridge that tells a customer site to rebuild when content changes

The CMS core — `src/routes/` and `src/views/`

This is the original content-management system each site gets: an **admin panel** (`/admin`) for writing posts, managing categories, media, menus, redirects, and SEO; a **public REST API** (`/api/public/v1/*`) that external automation uses to publish content; and the **frontend** router (sitemaps, RSS, robots.txt). It's

the mature, frozen part — the SaaS layer was built *around* it without changing it.

Part 7 — The customer site template (`site-template` repo)

This is the **cookie-cutter** — a complete, ready-to-build website made with **Astro**. When a customer provisions a site, GitHub copies this whole repo into their account and the build produces their live site. Key contents:

File / folder	Purpose
<code>astro.config.mjs</code> , <code>package.json</code>	The build setup
<code>src/pages/</code>	Every page type: posts, categories, shop, authors, locations, RSS, sitemaps, <code>llms.txt</code> (for AI), etc.
<code>src/layouts/Base.astro</code>	The shared page shell
<code>src/lib/cms.ts</code>	Pulls content from the CMS API at build time
<code>src/lib/presets.ts</code>	The design presets (colors/fonts) — must match the brain's list
<code>public/fonts/*.woff2</code>	The self-hosted fonts (7 of them) — these are the binary files that made a plain copy-paste impossible and needed a real <code>git push</code>
<code>.github/workflows/*.yml</code>	The build/deploy scripts — these are the files that require the token's <code>workflow</code> scope
<code>scripts/check-*.mjs</code>	Quality gates that <i>block a broken build</i> (SEO files present, fonts subset, contrast passes, zero-JS, correct headers)

That last point is why the template can't just be a few files: it enforces its own quality on every build, so customer sites can't ship broken.

Part 8 — What actually happens when you click "Create site"

The **provisioning pipeline** (`src/modules/provisioning/provisionSite.ts`) runs these steps in order. It's **resumable** — if a step fails, you fix the cause and hit "Retry from the failed step" and it picks up where it stopped.

1. **Create your content workspace** — makes the new site's private database.
2. **Set up spam protection** — Turnstile captcha (skips gracefully if not configured — that's the "SKIPPED" you saw, which is fine).
3. **Create the site repository in your GitHub** — copies `site-template` into the customer's account. *(This is the step that kept failing 404 → 422 because the template didn't exist / was empty / wasn't flagged as a template.)*
4. **Write the site configuration** — drops in the site's settings file.
5. **Store deploy credentials in your repository** — puts the needed secrets into the new repo so it can deploy itself.

6. **Start the first build** — triggers GitHub Actions to build the site.
7. **Confirm the site built and deployed** — waits for success.
8. **Connect your domain (apex + www)** — points the domain at the site.
9. **Turn off the workers.dev preview URL** — so only the real domain serves it.
10. **Enable Cloudflare protection** — security settings.
11. **Wire publish-to-rebuild** — so future content edits auto-rebuild the site.

Part 9 — The whole troubleshooting journey (what broke and why)

This is the story of everything we fixed, so you understand *why* each change mattered:

Symptom you saw	Real cause	The fix
Worker error mentioning <code>'execute'</code> ; signup dead	SaaS mode was off	Set <code>SAAS_MODE="1"</code> + <code>SAAS_APP_HOSTNAME</code>
"This site can't be reached" on the domain	DNS not pointed yet	Waited for <code>freecoinslink.de</code> nameservers to go live, used a real subdomain
404 "network admin panel"	Dashboard was pointed at the admin-only hostname	Moved dashboard to <code>app.freecoinslink.de</code>
Signup "Something went wrong"	Password hashing asked for 600,000 rounds; Cloudflare caps it at 100,000	Capped it in code (PR #100) + set the variable to 100000
GitHub showed "Temporarily unavailable"	<code>GITHUB_APP_SLUG</code> kept getting wiped by deploys	Baked the slug into <code>wrangler.toml</code> (PR #103)
GitHub install redirected to old <code>arsal.app</code>	Wrong Setup URL on the GitHub App	Pointed it at your domain
Design presets not clickable	A broken empty function	Rewrote the picker (PR #102)
Every mention of <code>arsal.app</code>	Leftover placeholder domain	Replaced with <code>freecoinslink.de</code> everywhere (PR #102)
Repo creation <code>404</code>	<code>ArsalR/site-template</code> didn't exist	Create it (you did — repo now exists)
Repo creation <code>422</code>	Template existed but was empty (push kept failing)	Push must succeed first
Push "remote rejected ... workflow scope"	Token lacked the <code>workflow</code> permission	Make the token with both <code>repo</code> + <code>workflow</code> (the pre-ticked link)

Symptom you saw	Real cause	The fix
Turso token pasted in chat	Accidental exposure	Rotated it (and a GitHub token) — always revoke leaked tokens

Where you are right now: the *only* remaining blocker is pushing the template with a `workflow`-scoped token. Once that push shows `main -> main`, the repo has content + is flagged as a template + your GitHub App can see it → "Retry" clears, and provisioning continues to the Cloudflare deploy steps.

Part 10 — How to run it day-to-day (cheat sheet)

Always run commands **from the project folder**: `cd ~/pinterest-cms`.

```
npm run typecheck    # check the code compiles (before deploying)
npm test             # run the automated tests
npm run deploy       # push the brain live to Cloudflare
npx wrangler secret list                # see which secrets are set (not their values)
npx wrangler secret put NAME_OF_SECRET # set/replace a secret (paste into the hidden prompt)
```

Rules of thumb:

- **Never paste a real token/password** into chat, email, or a screenshot.
- After `git pull`, the `wrangler.toml` values win — don't hand-edit them; change them in the file and commit.
- If a provisioning step fails, read the red line: it names the step and a code (404 = missing, 422 = exists-but-invalid, 403 = permission). Fix that, hit Retry.

Glossary (the jargon, decoded)

- **Worker** — a small program Cloudflare runs for you, worldwide, on demand.
- **Repo (repository)** — a folder of code stored on GitHub, with history.
- **Template repo** — a repo flagged so GitHub can *copy* it into new repos.
- **Token / secret / API key** — a password for computers.
- **Scope / permission** — what a token is *allowed* to do.
- **Provisioning** — the automated setup of a new customer site.
- **Astro** — the tool that turns the template into a fast static website.
- **Turso / libSQL / SQLite** — the database technology (one per site).
- **R2** — Cloudflare's file storage.
- **DNS / nameservers** — the internet's address book pointing a domain to a server.
- **PBKDF2** — the math that safely scrambles passwords.
- **Idempotency** — "doing it twice has the same effect as doing it once."
- **BYO infra** — customers use *their own* GitHub/Cloudflare, so they own it.

You built a genuine multi-tenant website factory. The hard engineering is done; what's left is the one-time bootstrap (the template repo) and connecting accounts — plumbing, not construction. You're at the finish line.